

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ  
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: Генетические алгоритмы**

|                  |       |               |
|------------------|-------|---------------|
| Студент гр. 4345 | _____ | Е.В. Гаврилов |
| Студент гр. 4345 | _____ | С.А. Кравцов  |
| Студент гр. 4345 | _____ | Д.Н. Бабий    |
| Руководитель     | _____ | Т.Р. Жангиров |

Санкт-Петербург  
2026

## ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: Д.Н. Бабий, С.А. Кравцов, Е.В. Гаврилов

Группа: 4345

Тема практики: Генетические алгоритмы

Задание на практику:

Задача о порядке перемножения матриц

Дано  $N$  матриц  $A$  с произвольными размерами ( $A_1: p_0 \times p_1, A_2: p_1 \times p_2, \dots, A_N: p_{N-1} \times p_N$ ). Необходимо определить порядок перемножения матриц, чтобы минимизировать количество операций умножения для вычисления результирующей матрицы  $B: p_0 \times p_N$

Сроки прохождения практики: 06.07.2026 – 18.06.2026

Дата сдачи отчета: 17.06.2026

Дата защиты отчета: 17.06.2026

Студент гр. 4345

Д.Н Бабий

Студент гр. 4345

С.А Кравцов

Студент гр. 4345

Е.В Гаврилов

Руководитель

Т.Р. Жангиров

## **АННОТАЦИЯ**

В ходе учебной практики разработана программа для решения задачи оптимального порядка перемножения цепочки матриц с использованием генетического алгоритма. Реализованы алгоритм поиска, графический интерфейс для настройки параметров и ввода данных, а также средства визуализации процесса работы и результатов. Программа позволяет сравнивать качество найденных решений с точным и жадным алгоритмами и демонстрирует применение генетических алгоритмов для решения задач дискретной оптимизации.

## **SUMMARY**

During the training practice, a program was developed to solve the problem of finding the optimal order for multiplying a chain of matrices using a genetic algorithm. The program includes a search algorithm, a graphical interface for setting parameters and entering data, and tools for visualizing the process and results. The program allows users to compare the quality of the solutions found with exact and greedy algorithms and demonstrates the use of genetic algorithms for solving discrete optimization problems.

## СОДЕРЖАНИЕ

|                                                 |    |
|-------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                   | 5  |
| 1 ПОСТАНОВКА ЗАДАЧИ.....                        | 6  |
| 1.1. Предметная область.....                    | 6  |
| 1.2. Задачи практики.....                       | 6  |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ.....                        | 8  |
| 2.1. Генетический алгоритм.....                 | 8  |
| 2.2. Выбор технологий, распределение ролей..... | 9  |
| 2.3. Реализация генетического алгоритма.....    | 10 |
| 2.4. Реализация GUI.....                        | 11 |
| 2.5. Визуализация работы алгоритма.....         | 13 |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....                  | 15 |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ.....                       | 16 |
| ЗАКЛЮЧЕНИЕ.....                                 | 17 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....           | 18 |
| ПРИЛОЖЕНИЕ А Диаграмма классов.....             | 19 |

## **ВВЕДЕНИЕ**

Предметной областью данной практики являются методы оптимизации вычислений и эвристические алгоритмы поиска решений. Рассматривается классическая задача минимизации вычислительных затрат, имеющая прикладное значение в различных сферах: от обработки данных до численного моделирования. Особое внимание уделяется генетическим алгоритмам как инструменту решения дискретных оптимизационных задач, где пространство возможных решений велико, а применение точных методов затруднено или нецелесообразно.

Цель практики — закрепление навыков проектирования и реализации программных систем с графическим интерфейсом, а также освоение принципов работы эволюционных алгоритмов через их практическое применение к задаче поиска оптимальной структуры вычислений.

Задачи практики включают изучение теоретического материала, самостоятельную реализацию алгоритмического ядра и визуализации, организацию пользовательского интерфейса для управления ходом вычислений, а также обеспечение возможности анализа и сравнения получаемых результатов. Отдельное внимание уделяется средствам взаимодействия с данными и документированию выполненной работы.

# **1 ПОСТАНОВКА ЗАДАЧИ**

## **1.1. Предметная область**

Задача о порядке перемножения матриц актуальна для областей, требующих высокопроизводительных вычислений: машинное обучение, компьютерная графика, САПР, оптимизация запросов в базах данных. Различные варианты расстановки скобок дают одинаковый результат, но количество скалярных операций может отличаться на порядки. Классический метод решения — динамическое программирование. Использование генетического алгоритма в данной работе позволяет исследовать поведение эвристических методов на дискретных оптимизационных задачах, визуализировать процесс эволюции популяции решений и получить навыки реализации алгоритмов без готовых библиотек.

## **1.2. Задачи практики**

1. Изучить математическую постановку задачи перемножения матриц и принципы работы генетических алгоритмов (выполнил: Степан Кравцов, как ответственный за реализацию алгоритма).

2. Выбрать язык программирования, библиотеку графического интерфейса и прочие используемые технологии, сформировать бригаду, распределить роли (выполнил: Дмитрий Бабий как лидер команды проекта).

3. Реализовать вычислительное ядро (функция качества, кодирование решений, операторы селекции, кроссовера и мутации) без использования готовых библиотек ГА (выполнил: Степан Кравцов, как ответственный за реализацию алгоритма).

4. Разработать GUI с возможностью ввода данных вручную, загрузки из файла и случайной генерации, обеспечить настройку параметров алгоритма пользователем, возможность пропуска шагов и перехода к конечному результату и откат назад (выполнили: Гаврилов Евгений, как ответственный за реализацию графического интерфейса, Дмитрий Бабий, как ответственный за визуализацию работы алгоритма).

5. Реализовать пошаговую визуализацию: текущая аппроксимация, оптимальное решение, график функции качества от поколения (выполнил: Дмитрий Бабий, как ответственный за визуализацию работы алгоритма).

6. Подготовить отчет по проделанной работе (выполнил: Гаврилов Евгений, Дмитрий Бабий, (в т.ч. диаграмма классов, как ответственный за визуализацию), Степан Кравцов, каждый член команды выполнил отчет по части, за которую отвечает, и коллективно — то, что касается организационной части).

## **2 РЕЗУЛЬТАТЫ РАБОТЫ**

### **2.1. Генетический алгоритм**

Генетические алгоритмы — это алгоритмы, имитирующие процессы естественного отбора и воспроизводства и решающие оптимизационные, поисковые задачи и задачи обучения [1,2]. Их преимущество заключается в том, что они применимы к задачам, имеющим сложную математическую модель или не имеющим ее [1].

Реализуемый генетический алгоритм решает задачу минимизации числа скалярных умножений при перемножении цепочки матриц.

Решение (особь) кодируется как список целых чисел — на каждом шаге умножения ген указывает индекс матрицы в текущем списке размерностей, которая будет перемножена со следующей. Длина хромосомы равна  $N-2$ , где  $N$  — количество матриц. Такой способ гарантирует корректность любого случайно сгенерированного решения без дополнительной валидации.

#### **Кодировка индивида:**

Решение (особь) кодируется последовательностью целых чисел длиной  $N-2$ . Каждый ген определяет индекс матрицы в текущем списке размерностей, которая будет перемножена со следующей матрицей. После выполнения умножения список размерностей обновляется, поэтому одинаковое значение гена на разных этапах может соответствовать различным матрицам. Такая кодировка гарантирует корректность любого случайно сгенерированного решения и не требует дополнительной проверки допустимости.

#### **Функция приспособленности:**

Подсчет количества операций необходимых для перемножения матриц в порядке, заданном хромосомами индивида.

#### **Целевая функция:**

Если  $N \leq 30$ , тогда целевой функцией является фактическое значение минимального количества операций для перемножения матриц ( $\text{min\_cost}$ ) полученное за  $O(n^3)$  и является глобальным минимумом, для  $N = 30$ ,



количество операций потраченное на целевую функцию  $27000 \ll 300000$  операций алгоритма с популяцией в 100 особей и 100 поколениями, для меньших  $N$  — аналогично. Поэтому оценка оправдана. На практике генетический алгоритм может не достигнуть глобального оптимума, однако в большинстве проведённых экспериментов получаемое решение отличалось от оптимального не более чем на 10-20%.

Далее, при росте  $N$ , из-за сложности  $O(N^3)$  количество операций, потраченное на расчет перебором будет стремительно расти, быстрее чем количество операций алгоритма. Поэтому:

Для  $N > 30$  точное решение вычислять нецелесообразно. В качестве верхней оценки используется результат жадного алгоритма со сложностью  $O(n^2)$ . Полученное генетическим алгоритмом решение сравнивается с этой оценкой. Если стоимость решения меньше стоимости, полученной жадным алгоритмом, то генетический алгоритм показал более качественный результат. Отношение стоимости решения генетического алгоритма к верхней оценке используется как показатель качества полученного решения.

## **2.2. Выбор технологий, распределение ролей**

Для написания программы выбран язык Python3, поскольку с этим инструментом знакомы все члены команды и для него существует большое количество библиотек для создания графического интерфейса. Для GUI выбрана библиотека PySide6, так как она является портом библиотеки Qt на Python3, наиболее портируема среди аналогов и подробно документирована [3,4].

Роли в команде распределены следующим образом:

- Степан Кравцов — реализация генетического алгоритма
- Евгений Гаврилов — графический интерфейс
- Дмитрий Бабий — связь графического интерфейса и алгоритма, инфографика

## 2.3. Реализация генетического алгоритма

Реализован класс **GeneticAlgorithm**, инкапсулирующий размерности матриц, текущую популяцию и историю поколений.

### Основные компоненты:

- `_generate_individual` — создаёт случайную особь, где каждый ген  $g_i$  лежит в диапазоне  $[0, \text{ind\_size}-1-i]$ ,  $\text{ind\_size} = N-2$ .
- `_calculate_cost` — вычисляет функцию приспособленности: последовательно перемножает матрицы согласно порядку, заданному хромосомой, суммируя затраты  $p_a \cdot p_b \cdot p_c$  на каждом шаге.
- `get_min_cost` / `get_greedy_cost` — обёртки над функциями `calculate_min_cost` (динамическое программирование, эталонный глобальный минимум) и `greedy_cost` (жадный алгоритм, на каждом шаге выбирающий пару соседних матриц с минимальным числом операций) из модуля `helpers`.
- `_tournament` — турнирная селекция: из `tournament_size` случайно выбранных особей (по умолчанию 3, настраивается через `set_tournament_size`) побеждает особь с наименьшей стоимостью.
- `_crossover` — одноточечное скрещивание, порождающее двух детей; корректность потомков обеспечивается одинаковой структурой допустимых значений генов на соответствующих позициях.
- `_mutate` — независимая мутация каждого гена с вероятностью  $p_m$ , заменяющая ген на случайное допустимое значение. По умолчанию  $p_m = 1/(N-2)$ , то есть в среднем на особь приходится одна мутация; значение может быть переопределено пользователем через панель параметров.
- `_selection` — формирует новое поколение: попарно отбирает родителей турниром, с вероятностью  $p_c$  скрещивает их, к потомкам применяет мутацию.
- `evolution(steps)` — при пустой популяции генерирует стартовую популяцию, затем на каждом шаге вычисляет стоимости всех особей,

фиксирует наименьшую и среднюю стоимость поколения, сохраняет снимок в `history`, выполняет селекцию и применяет элитизм — принудительно возвращает в новую популяцию оптимальную особь предыдущего поколения на случайную позицию, что гарантирует немонотонное неухудшение найденного решения.

- `go_next_generate` — выполняет одну итерацию вперёд;
- `degradation(steps) / go_prev_generate` — уменьшает номер текущего поколения и восстанавливает популяцию из сохранённого снимка `history[cur_generation]`, не пересчитывая эволюцию заново;
- `go_first_generate` — откатывает к первому сохранённому поколению;
- `go_last_generate(n)` — доводит эволюцию до заданного номера поколения `n`, довычисляя недостающие шаги через `evolution`.

### **Связь с GUI**

Интерфейс получает от пользователя размерности матриц одним из трёх способов (случайная генерация, загрузка из файла, ручной ввод) и параметры алгоритма (размер популяции, число поколений, вероятности мутации и скрещивания). Список пар размерностей приводится к единому списку `dimensions` функцией `pairs_to_dimensions` и передаётся в `GeneticAlgorithm`. Далее объект алгоритма настраивается методом `set_params` и запускается методом `evolution`. Результат для отображения формируется методом `get_plot_data`, который собирает накопленную историю в структуру `PlottingData` (номера поколений, целевая и жадная оценки, наименьшая/средняя стоимость по поколениям, порядок умножения оптимального решения).

Структура компонентов алгоритма и связи алгоритма с графическим интерфейсом изображены на рис. 1 и рис. 2 в Приложении А.

## **2.4. Реализация GUI**

Графический интерфейс реализован с использованием библиотеки `PySide6`. Главное окно `MainWindow` задаёт заголовок и минимальный размер

окна и устанавливает центральным виджетом `DashboardPage`. Диаграмма классов главного окна приведена на рис. 4.

Центральный виджет `DashboardPage` разбит сплиттером на левую и правую панель.

#### **Компоненты левой панели:**

- `ChoiceBoard` — выбор источника матриц через `QButtonGroup` с тремя вариантами (`Source.RANDOM`, `Source.FILE`, `Source.MANUAL`): случайная генерация заданного количества матриц (`QSpinBox`, диапазон 3–1000) через `get_random_matrices`; загрузка из текстового файла через `QFileDialog` и `get_data_from_file` с валидацией согласованности размерностей (несовпадение вызывает `ValueError` и выводится в статус-строку красным); ручной ввод через отдельный диалог `ManualInputDialog`. Полученный набор матриц выводится в `QListWidget` и транслируется сигналом `matrices_changed`.
- `ManualInputDialog` — модальный диалог пошагового добавления матриц: пользователь вводит высоту и ширину, кнопка «+» активна только при валидных значениях (`QIntValidator`), при добавлении проверяется совместимость с последней (или первой) матрицей цепочки, кнопка «Готово» доступна только при наличии не менее двух матриц.
- `GAParamsPanel` — форма (`QFormLayout`) с параметрами алгоритма (размер популяции, число поколений, вероятности мутации и скрещивания), возвращаемыми в виде датакласса `Params`. Подписи полей реализованы через `HintLabel` с подсказкой (`toolTip`) и пунктирным подчёркиванием, отрисовываемым в `paintEvent`.
- Управляющие кнопки: «Запустить алгоритм» (запуск/дозапуск эволюции с текущими параметрами), «Значения по умолчанию» (сброс параметров).

Диаграмма классов ввода приведена на рис. 3.

### **Компоненты правой панели:**

- график сходимости на основе `pyqtgraph.PlotWidget`;
- панель навигации по поколениям (первое / предыдущее / следующее / последнее поколение);
- `ResultsPanel` — сводка результатов (наименьшая стоимость, отношение к целевому значению, номер поколения, на котором достигнут наименьший результат, порядок умножения) с возможностью выделения текста и кнопкой «Показать дерево умножения», открывающей диалог визуализации.

## **2.5. Визуализация работы алгоритма**

Разработаны две формы визуализации результатов работы алгоритма: график сходимости и дерево найденного порядка перемножения цепочки матриц. Диаграмма классов визуализации приведена на рис. 1.

### **График сходимости**

`pyqtgraph.PlotWidget` строит четыре кривые в относительных координатах «отношение к целевой стоимости» от номера поколения: наименьшая стоимость поколения и средняя стоимость поколения (обе — как отношение точного минимума `target_cost` к текущему значению), горизонтальная линия целевого отношения, равная 1 (соответствует достижению точного минимума), и горизонтальная линия верхней оценки, полученной жадным алгоритмом ( $\text{target\_cost} / \text{greedy\_cost}$ ), что показывает, насколько решение генетического алгоритма приближается к оптимуму и превосходит ли оно жадную эвристику.

С помощью механизма навигации реализовано пошаговое представление алгоритма: каждое переключение шага вызывает соответствующий метод `GeneticAlgorithm` (`go_first_generate`, `go_prev_generate`, `go_next_generate`, `go_last_generate`) и перерисовывает график и панель результатов актуальными данными из `get_plot_data`.

### **Визуализация порядка умножения**

Диалог MultOrder строит дерево умножения на основе QGraphicsScene/QGraphicsView. Исходные матрицы отображаются в один ряд как листовые узлы (TreeNode, синий цвет), затем для каждого шага хромосомы создаётся узел слияния (оранжевый), соединенный кривыми Безье с двумя узлами; последний шаг помечается как корень дерева (зелёный). Метод `get_perf_point` вычисляет точку пересечения линии с границей эллипса узла, так что соединительные кривые подходят точно к контуру узла, а не к его центру.

Диаграмма классов реализованной программы приведена на рис. 5.

### **3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ**

В качестве языка проекта использован Python3. В проекте использована стандартная библиотека Python. Для реализации графического интерфейса использована библиотека PySide6. Диаграммы классов построены с помощью plantuml. Разработка программы проведена с использованием системы контроля версий git, репозиторий размещен на сайте github.com [5].

#### **4 ОТЗЫВ РУКОВОДИТЕЛЯ**

По результатам работы учебная практика оценена на 5.



## ЗАКЛЮЧЕНИЕ

В ходе выполнения практики была разработана программа для решения задачи об оптимальном порядке перемножения цепочки матриц с использованием генетического алгоритма.

Изучена математическая постановка задачи о порядке перемножения матриц, основные принципы работы генетических алгоритмов — кодирование решений, операторы селекции, кроссовера и мутации, механизм элитизма. Выбран способ кодирования особи и обоснования целевой функции для разных диапазонов  $N$ .

Выбран ЯП Python 3 и библиотека PySide6 для реализации графического интерфейса. Сформирована бригада, распределены роли.

Разработан эволюционный алгоритм. Для оценки качества получаемых решений реализован точный метод динамического программирования (используется при  $N \leq 30$  как глобальный минимум) и жадный алгоритм (используется как верхняя оценка при  $N > 30$ , когда точный расчёт становится вычислительно нецелесообразным).

Разработан интерфейс, позволяющий вводить данные вручную (с пошаговой проверкой согласованности размерностей), из текстового файла и путём случайной генерации. Реализована настройка ключевых параметров алгоритма пользователем (размер популяции, число поколений, вероятности мутации и скрещивания). Реализована возможность пошагового перехода между поколениями в обе стороны, а также переход к конечному результату.

Реализована пошаговая визуализация процесса эволюции: график изменения наименьшей и средней стоимости популяции по поколениям в относительных координатах (отношение к целевому значению), с отображением как точного оптимума, так и верхней оценки жадного алгоритма для наглядного сравнения качества решения. Реализована визуализация оптимального порядка умножения в виде дерева слияний с использованием Qt.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Вирсански Э. Генетические алгоритмы на Python. - М.: ДМК Пресс, 2020.
2. Панченко Т.В. Генетические алгоритмы. - Астрахань: Издательский дом «Астраханский университет», 2007.
3. Qt Documentation URL: <https://doc.qt.io/> (дата обращения: 15.07.2026).
4. PySide documentation URL: <https://pyside.readthedocs.io/en/latest/> (дата обращения: 15.07.2026).
5. Генетический алгоритм минимизации числа скалярных умножений при перемножении цепочки матриц // [github.com](https://github.com) URL: <https://github.com/kravtsov07/team-work/> (дата обращения: 15.07.2026).

## ПРИЛОЖЕНИЕ А

### ДИАГРАММА КЛАССОВ

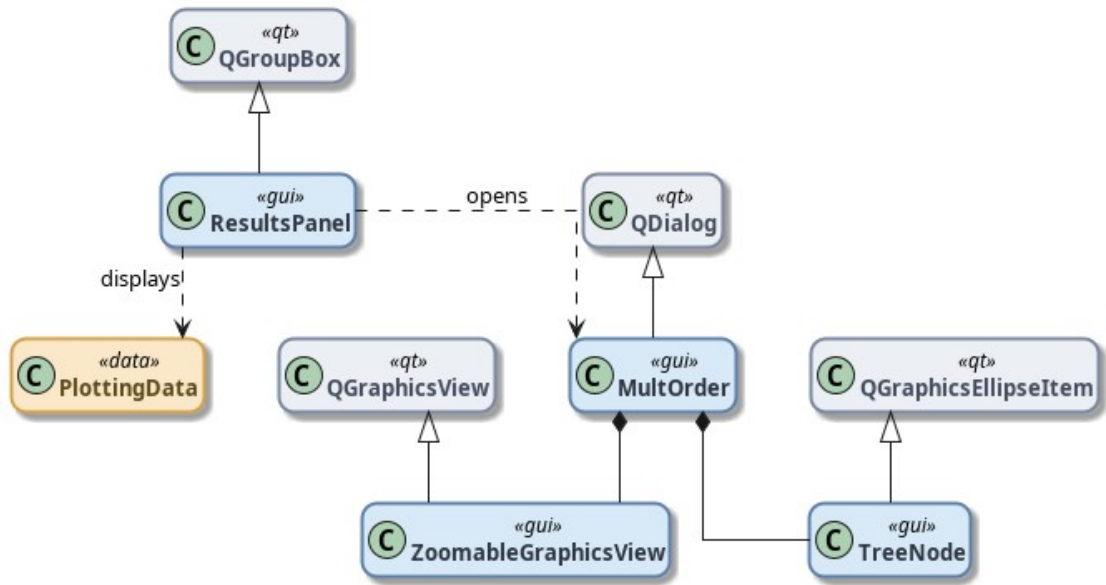


Рисунок 1 — Диаграмма классов визуализации

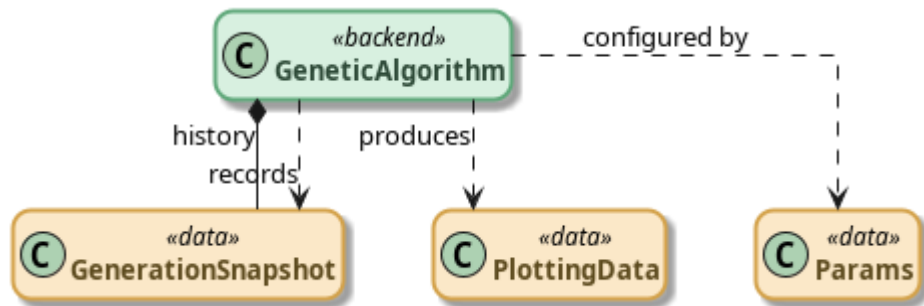


Рисунок 2 — Диаграмма классов backend

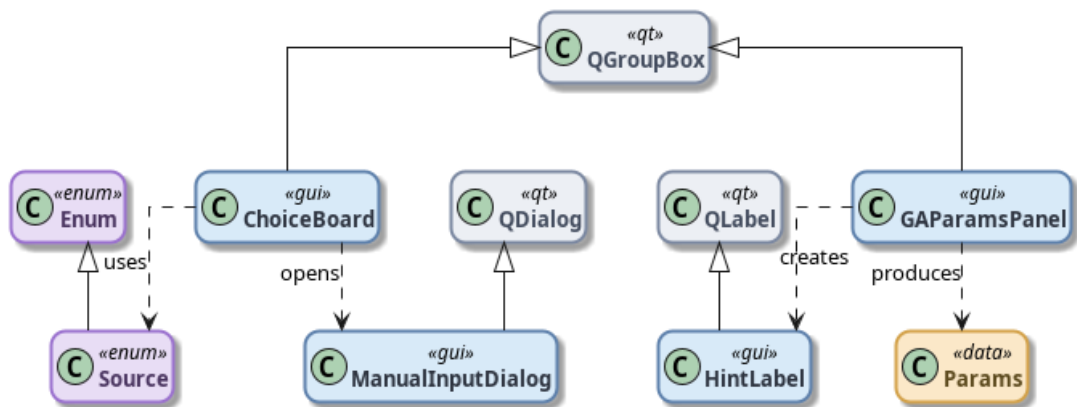


Рисунок 3 — Диаграмма классов ввода данных и параметров

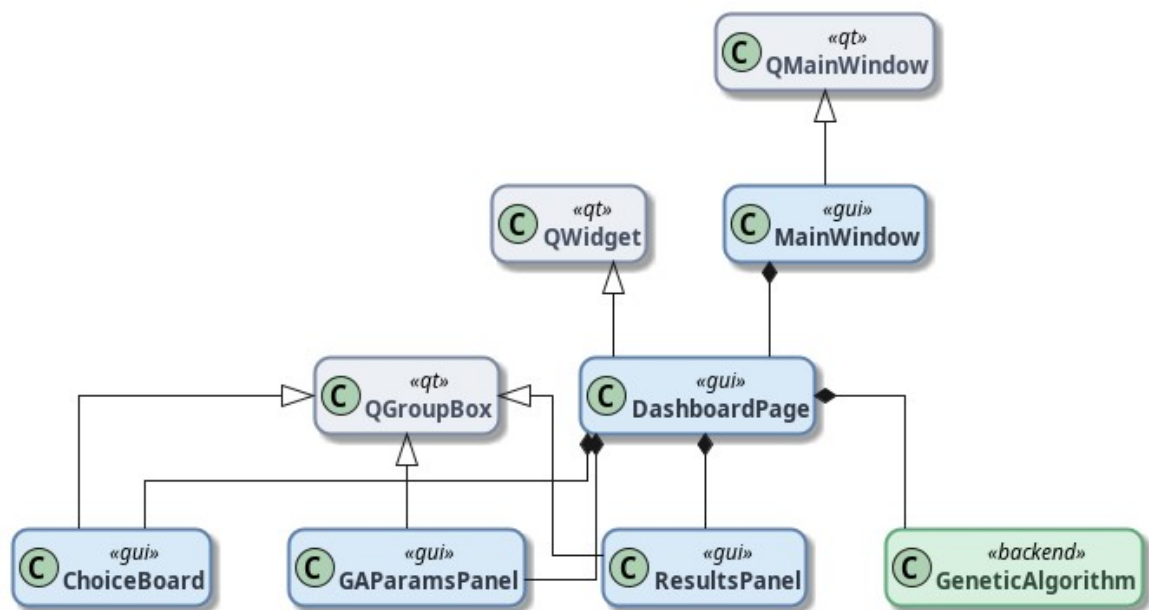


Рисунок 4 — Диаграмма классов главного окна

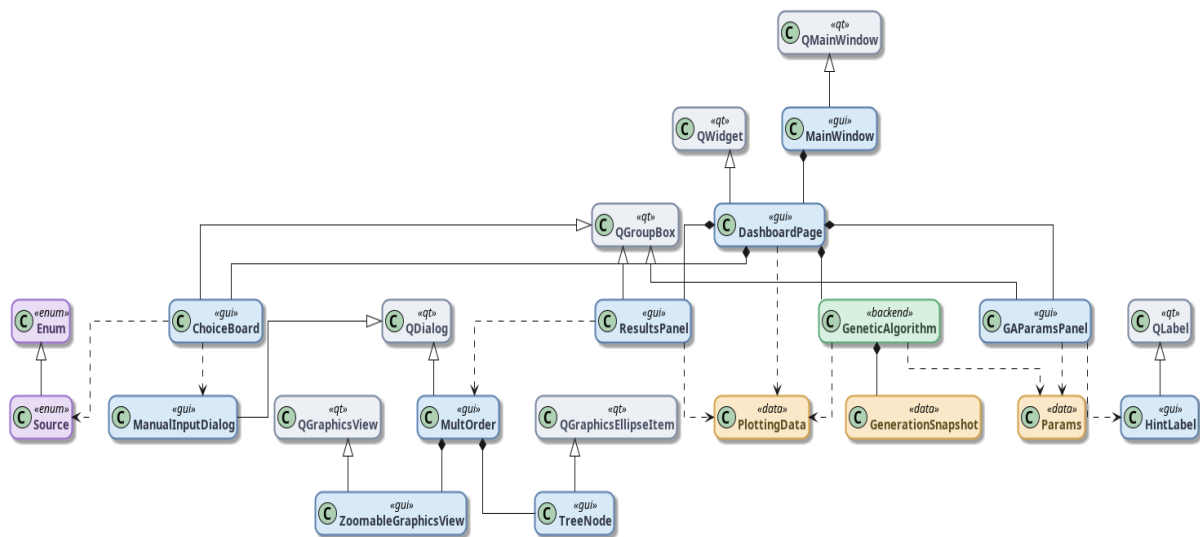


Рисунок 5 — Общая диаграмма классов программы